# LAB ASSIGNMENT # 03

## Linear Loop




**CSE110 | Programming Language I**

| | LAB TASKS | HOME TASKS |
|---|---|---|
| CODING | 07 | 08 |
| TRACING | 01 | 02 |

**NOTE: You need to submit only the Home Tasks. Submit all the Flowchart or Tracing tasks hand drawn or handwritten respectively to your Lab Instructors before the next lab. Submit all the Homework Coding Tasks in the Google Form shared on buX.**

**LAB TASKS**

**[NO NEED TO SUBMIT]**

**Question: 1**

Write a Java code that would print the following sequences using while/for loop:

I. **24, 18, 12, 6, 0, -6**
II. **18, -27, 36, -45, 54, -63**

**Question: 2**

Write a Java program that will take N numbers from the user and find their sum and average using a for loop.

Sample Input:
N = 5
Input the 5 numbers:
1
2
3
4
5
Expected Output:
The sum of 5 no is: 15
The Average is: 3.0

**Question: 3**

You are the engineer maintaining the power distribution system of NeoVolt City, a futuristic city powered by solar grids.

Each day the power grid generates and loses energy based on environmental conditions:

- If the day number is divisible by 3, a solar storm occurs, power drops by 45 units
- If divisible by 5, wind turbines overproduce, power increases by 25 units
- Otherwise, normal operation, power increases by 10 units

The grid starts with 40 units of power.

If the power ever exceeds 500, excess power is wasted and reduced by 50 units to protect it from overloading.

If power falls below 0, a blackout occurs, stop the simulation immediately.

| Sample Input | Sample Output |
|---|---|
| Enter Number of Days: 5 | Final Power: 50 |

| | |
|---|---|
| Enter Number of Days: 12 | Blackout on Day 9 |

**Question: 4**

Write a Java program that will keep taking integer numbers as inputs from the user and print the square of those numbers until it gets a negative number and then stop.

Sample Input/Output: (The purple numbers are input.)
Enter Number: 2
2 ^ 2 = 4
Enter Number: 6
6 ^ 2 = 36
Enter Number: 1
1 ^ 2 = 1
Enter Number: 4
4 ^ 2 = 16
Enter Number: -5

**Question: 5**

Write a Java program that will calculate the value of y where the expression of y is as follows :

$$y = 1^2 - 2^2 + 3^2 - 4^2 + 5^2 ..... + n^2$$

| Sample Input | Output | Explanation |
|---|---|---|
| 5 | 15 | y = 1 - 4 + 9 - 16 + 25 = 15 |
| 10 | -55 | y = 1 - 4 + 9 - 16 + 25 - 36 + 49 - 64 + 81 - 100 = -55 |
| 20 | -210 | y = 1 - 4 + 9 - 16 + 25 ……. - 400 = -210 |

**Question: 6**

Write a program in Java that asks the user for an **integer** input and counts the number of digits in the number.
**Hint:** You may keep dividing the number by ten and count how many times this can be done until the number becomes 0.

| Sample Input | Output |
|---|---|
| 7546 | Total digits = 4 |

**Question: 7**

```
1  public class Trace1{
2    public static void main(String args[]){
3        int x = 0, p = 0, sum = 0;
4        p = 1;
5        x = 2;
6        double q;
7        sum = 0;
8        while (p < 12){
9           q =  x + p -(sum+7/3)/3.0%2 ;
10          sum = sum + (++x) + (int)q;
11          System.out.println(sum++);
12          if (x > 5){
13             p += 4/2;
14          }
15          else {
16             p += 3%1;
17          }
18       }
19       sum = sum + p;
20       System.out.println(sum);
21   }
22 }
```

## HOME TASKS

**Question: 1**

Write a Java program that will take a positive integer *n* as input and print all the numbers from 0 to *n* which are **divisible by 5 but not divisible by 3**.

**Sample Input**
40
**Sample Output**
5
10
20
25
35
40

**Question: 2**
In Hogwarts, apprentice wizards brew magic potions using glowing herbs and sparkling crystals collected from the Whispering Woods. You have a stash of H herbs and C crystals. Each successful potion requires exactly 3 herbs and 2 crystals. Brew as many potions as possible without running short mid-mix stop when you can't complete another full potion. After brewing, check the total potions: if even, the batch is perfectly balanced ("Stable Elixir"); if odd, it's unpredictably fizzy ("Volatile Brew"). Output the number of potions and the balance message.

| Sample Input | Sample Output |
|---|---|
| Please Enter number of Herbs (H): 25<br>Please Enter number of Crystals (C): 13 | Potion-1 created<br>Remaining Herbs: 22, Remaining Crystals: 11<br>Potion-2 created<br>Remaining Herbs: 19, Remaining Crystals: 9<br>Potion-3 created<br>Remaining Herbs: 16, Remaining Crystals: 7<br>Potion-4 created<br>Remaining Herbs: 13, Remaining Crystals: 5<br>Potion-5 created<br>Remaining Herbs: 10, Remaining Crystals: 3<br>Potion-6 created<br>Remaining Herbs: 7, Remaining Crystals: 1<br>Potions Created: 6 |

| | Stable Elixir |
|---|---|
| Please Enter number of Herbs (H): 9<br>Please Enter number of Crystals (C): 6 | Potion-1 created<br>Remaining Herbs: 6, Remaining Crystals: 4<br>Potion-2 created<br>Remaining Herbs: 3, Remaining Crystals: 2<br>Potion-3 created<br>Remaining Herbs: 0, Remaining Crystals: 0<br>Potions Created: 3<br>Volatile Elixir |

**Question: 3**

In a sunny mountain cave, a baby dragon named Charmander plays a fun game to evolve. Charmander starts with E energy points. He must blow fire across N small rivers. For each river i, the distance is D units. If D is 5 or less, it costs 2 energy. If D is more than 5, it costs D / 2 energy (whole numbers only). Take away the energy after each blow. If energy goes below 0, Charmander gets tired and stops. Tell the river where he stops, or if he finishes all, say how much energy is left.

| Sample Input | Sample Output |
|---|---|
| Energy (E): 15<br>Number of River (N): 3<br>Enter River Distance D1: 4<br>Enter River Distance D2: 6<br>Enter River Distance D3: 3 | All Done<br>8 energy Left |
| Energy (E): 19<br>Number of River (N): 8<br>Enter River Distance D1: 14<br>Enter River Distance D2: 12<br>Enter River Distance D3: 9<br>Enter River Distance D4: 2<br>Enter River DIstance D5: 15 | Tired at River 5 |
| Explanations:<br>Energy is 19 and needs to blow fire across 8 rivers<br>In first river distance (D) is 14 which is greater than 5 so it costs 7 (half of D) so remaining energy (19-7=12)<br>In second river distance (D) is 12 which is greater than 5 so it costs 6 (half of D) so remaining energy (12-6=6)<br>In third river distance (D) is 9 which is greater than 5 so it costs 4 (half of D) so reaming energy (6-4=2) | |

In fourth river distance (D) is 2 which is less than 5 so it costs 2 so remaining energy (2-2=0)
In fifth river distance (D) is 15, but there is no energy left, therefore it got tired at River 5

**Question: 4**

Write a program in Java that prints the individual digits of an integer number (user input) backward (From right to left).

| Sample Input | Output |
|---|---|
| 32768 | 8, 6, 7, 2, 3 |

**Hint:** First to get the digit from the right side, we can take the remainder of the number using the modulus (%) operator i.e. mod 10 to get the rightmost digit and print it. For dropping the last digit, we can perform division by 10 on the number and then continue the same to print the other digits as shown below.

**Explanation:**

32768 % 10 = 8
32768 / 10 = 3276
Then,
3276 % 10 = 6
3276 / 10 = 327
and so on
327 % 10 = 7
327 / 10 = 32
32 % 10 = 2
32 / 10 = 3
3 % 10 = 3
3 / 10 = 0
Done! When the number becomes 0 you can end your loop.

**Question: 5**

In a secret city, you are on a mission to open a high-tech vault that protects a treasure!
The vault has a N-digit secret code (like 32768). To open it, you must say each digit one by one, from left to right, exactly as shown on the screen. Your smart watch can read the full number, but you need to break it into digits and speak them out loud like:
"3... 2... 7... 6... 8..."
If you get even one digit wrong, the alarm goes off!
**Write a Java program that:**

- Asks you: "Enter the N-digit vault code:"
- Reads the number
- Prints the digits forward, separated by commas

**Rules:**

- You cannot use **Math.pow()**
- You must count the digits first
- Then use division and remainder to pull out each digit
- Print them from left to right

[**Instructions From Headquarters:**
Start by finding how many digits (N). For 32768 → 5 digits. Then start with 10,000 (that's 10^(N-1)).
Divide the code by 10,000 → get 3
Take remainder → left with 2768
Now divide the divider by 10 → 1000
Repeat: divide, print, remainder, shrink divider
Until you print all digits!]

**Question: 6**
Write a Java program that will take an integer as input and -

A. Find out if the number is a prime number or not.
B. Find out if the number is a perfect number or not.

[**Prime Number:** If a number has only two divisors, (1 and itself), then it is a prime number. Else, then it is not a prime number.
**Perfect Number:** A number is said to be a perfect number if the sum of its divisors, including 1 but not the number itself is equal to that number.]

**Sample Input**
6
**Sample Output**
6 is not a prime number
6 is a perfect number

**Question: 7**
Write a Java program which adds all numbers that are multiples of either 7 or 9 but not both, up to 600. Use only a single loop in your program. *[Ans: 39814]*

**Question: 8**
Write a Java code that asks an integer as input from the user and takes that many integer inputs. Your task is to count how many numbers are non-negative and negative.

**Sample Input:** (The purple numbers are input.)
Enter an integer: 9
Enter number 1: -8
Enter number 2: 33
Enter number 3: -100
Enter number 4: 10
Enter number 5: 0
Enter number 6: 5
Enter number 7: 10
Enter number 8: -4
Enter number 9: 4
**Sample Output:**
6 Non-negative Numbers
3 Negative Numbers

**Question: 9**

```
1  public class Trace2 {
2      public static void main(String[] args) {
3          int x = 0, p = 0, sum = 0;
4          p = 1;
5          x = 2;
6          double q = 0.0;
7          sum = 5;
8          while (p < 15) {
9            q = x + p - (sum + (int) (7 / 4)) / 3.0 % 2;
10           sum = sum + x + (int) q;
11           x += 1;
12           System.out.println(sum);
13           if (x > 5) {
14                p += (int) (5 / 2);
15           }
16           else {
17                p += 10 % 3;
18           }
19         }
20         sum = sum + p;
21         System.out.println(sum);
22     }
23 }
```

**Question: 10**

```
1  public class Trace3 {
2    public static void main(String[] args) {
3      int m = 17, n = 13, p = 1, sum = 30;
4      while(0 < p%10){
5         if(m % 10 == 0){
6            sum = sum * m % n + p / n ;
7         }
8         else{
9            if(m % 4 == 0){
10              sum += n + (--m);
11           }
12           else{
13              sum -= m--;
14           }
15        }
16        p+=1;
17        System.out.println(sum);
18     }
19     System.out.println(!(n%13 == 0) && !false || p>10);
20   }
21 }
```

UNGRADED TASK

**Question: 1**

In a sunny garden, a gardener waters plants for H hours to help them grow. He starts with W water units. Each hour, he uses 3 units per plant, but there are 5 plants. If the hour is hot (hour number is a multiple of 2), use 4 units per plant instead. Subtract water each hour. If the water goes below 0 in any hour, stop and say the hour. If all hours are done, say the remaining water.

| Sample Input | Sample Output |
|---|---|
| Enter Hour (H): 5<br>Enter Water Units (W): 10 | Stopped at Hour 1 |
| Enter Hour (H): 5<br>Enter Water Units (W): 100 | All Watered<br>15 Water Unit Left |
| Explanation:<br>Plants=5 (fixed)<br>Water=100 (Starting with 100 Water Units)<br>Hour-1 (odd), so 5 * 3 = 15, Remaining Water: 85<br>Hour-2 (even), so 5 * 4 = 20, Remaining Water: 65<br>Hour-3 (odd), so 5 * 3 = 15, Remaining Water: 50<br>Hour-4 (even), so 5 * 4 = 20, Remaining Water: 30<br>Hour-5 (odd), so 5 * 3 = 15, Remaining Water: 15 | |